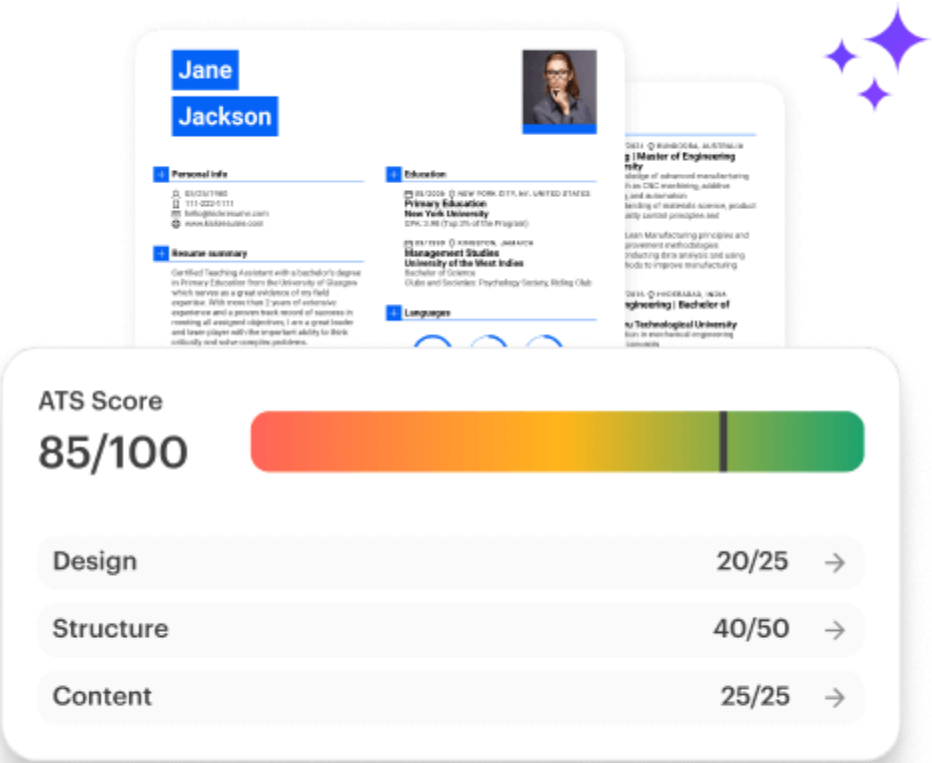
MBAI 5110G | Big Data System Design | Winter 2026

Project ATS Simulator

Resume Job Matching System

PySpark + MongoDB Atlas + Spark Structured Streaming

Jun De Guzman
Student ID: 100991306
Individual Project
March 22, 2026



1. Project Overview

This project builds a small, end-to-end Applicant Tracking System (ATS) simulation that scores resumes against job postings. The motivation was practical. A lot of applications get screened automatically, and I wanted to see what the scoring looks like when you strip it down to the core math (tokenization + TF-IDF + cosine similarity).

The pipeline processes **2,484 resumes** and **735 AI/data job postings**, generates similarity scores for every resume-job pair, and stores intermediate and final outputs in MongoDB Atlas. A Structured Streaming component then simulates "new resumes arriving" and writes the top matches back to MongoDB in near-real time.

Item	Details
Resumes / job postings	2,484 resumes (24 categories) and 735 AI/data science jobs
Resume-job pairs scored	604,891 total pairs (cosine similarity)
MongoDB collections	6 collections: raw, processed, match results, stream results
Parquet outputs saved	processed_resumes, processed_jobs, match_results
Streaming results	30 documents (10 resumes × top 3 matches each)
Benchmark scales	1×, 5×, 10× dataset replication (PySpark and Python baseline)
Platform	Google Colab, Spark 4.0.2, MongoDB Atlas (M0 Free)

2. Problem Statement and Datasets

Commercial ATS tools often filter applications long before a recruiter reads them. For students entering a crowded job market, that can feel like sending resumes into a black box. This project recreates a simplified version of that black-box step: given resume text and job-posting text, how does a basic scoring approach separate "likely match" from "likely reject"

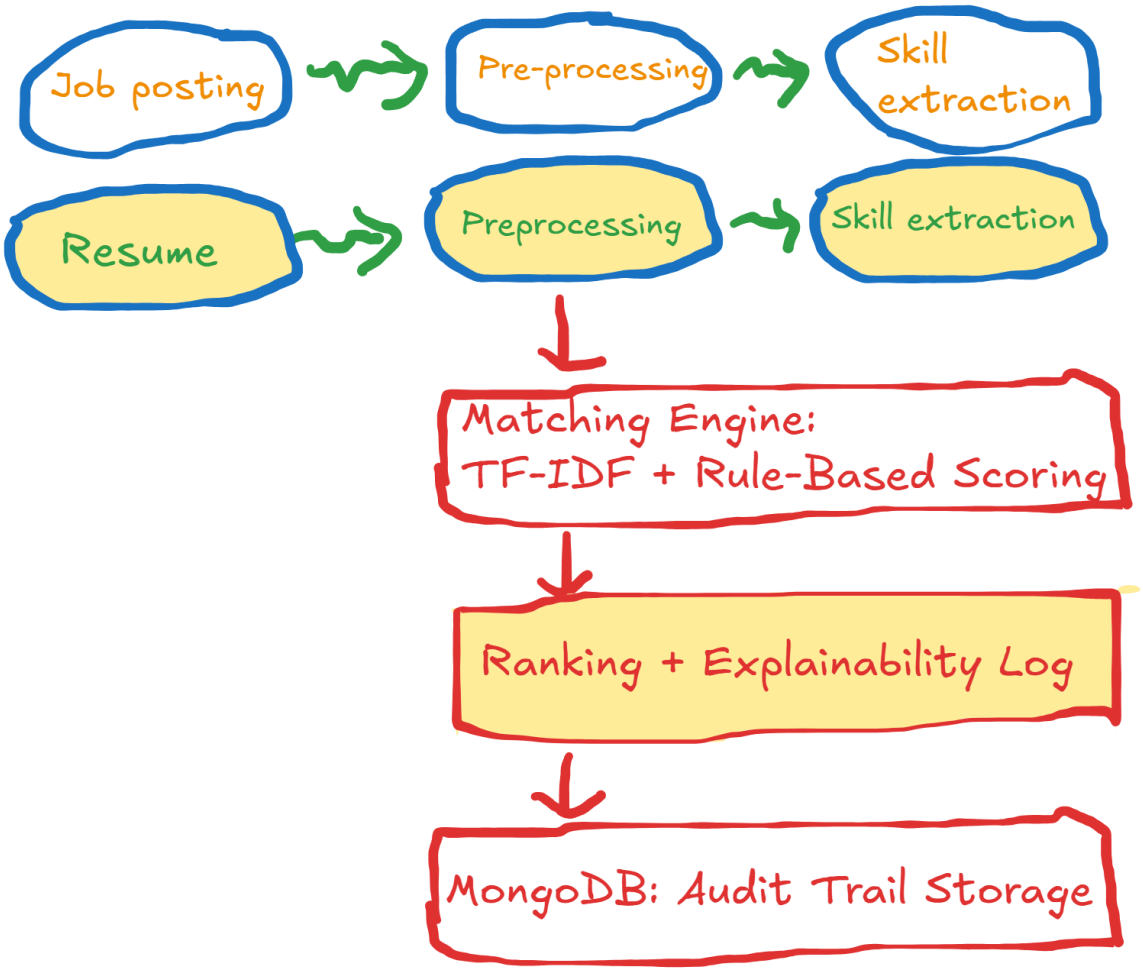
Datasets

Two public Kaggle datasets were used:

- Resume dataset (snehaanbhawal/resume-dataset): 2,484 resumes in CSV format (~54 MB), with an ID, plain-text resume, HTML resume, and one of 24 categories.
- Job postings dataset (princekhunt19/700-jobs-data-of-ai-and-data-fields-2025): 735 listings scraped for the search term "data scientist" (2025). Fields include title, company, description, location, and salary when available.

For ingestion, both datasets were cleaned in Pandas (HTML stripped, whitespace normalized, lowercased) and exported as JSON Lines. JSON Lines keeps each record self-contained, and it works cleanly with both Spark's read.json() and MongoDB bulk inserts.

3. Tools and Architecture



Tools Used	Version	Role in this project
Apache Spark	4.0.2 (PySpark)	Distributed processing for NLP, TF-IDF, cosine similarity, and Structured Streaming
Spark MLlib	Bundled with Spark	Tokenizer, StopWordsRemover, HashingTF, IDF pipeline stages
MongoDB Atlas	M0 Free (cloud)	Document storage for raw/processed data and match results
Python	3.12 (Colab)	Data prep/orchestration, small utilities, and visualizations
scikit-learn	via pip	Single-machine TF-IDF benchmark baseline
Matplotlib	via pip	Charts and outputs

System Architecture (batch + streaming)

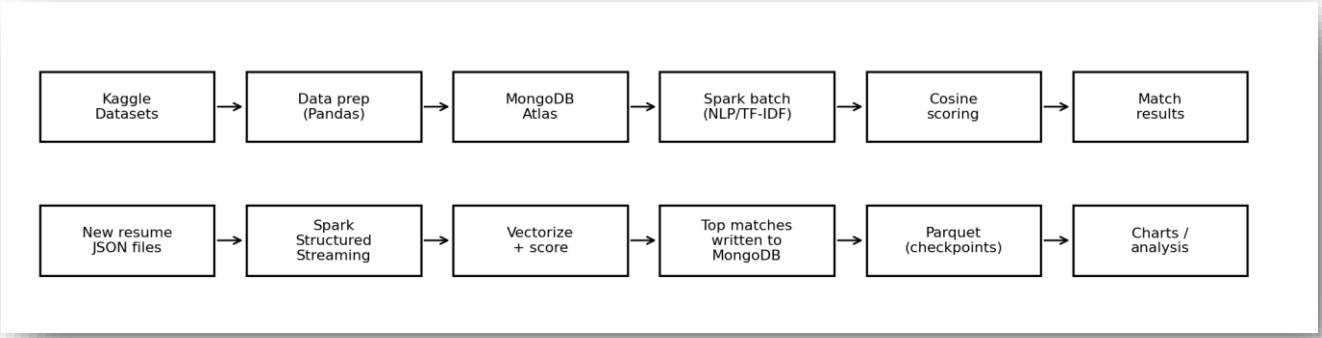


Figure 1. Batch pipeline (top) and streaming pipeline (bottom).

Key Processing Steps

Data preparation and MongoDB ingestion

Both CSV files were loaded in Pandas for a quick one-time clean (strip HTML tags, collapse whitespace, lowercase). At this scale (2,484 resumes), doing the initial scrub in Pandas is simply faster than paying Spark's startup overhead.

Cleaned records were inserted into MongoDB Atlas using PyMongo bulk upserts (ReplaceOne with upsert=True), so re-running the notebook does not create duplicates. Indexes were added on resume_id, category, job_id, and job_family to keep queries snappy.

PySpark batch processing

Spark read the JSON Lines files and applied a consistent text-preprocessing pipeline on both resumes and job postings:

- Lowercase and remove punctuation (regexp_replace)
- Tokenize text with MLlib Tokenizer
- Remove stopwords with StopWordsRemover
- Add a token_count column (size(tokens)) for quick sanity checks

Processed DataFrames were written back to MongoDB and also saved as Parquet. The Parquet files matter for benchmarking: they let later runs skip the preprocessing stage and start from a checkpoint.

TF-IDF vectorization and cosine similarity

A four-stage MLlib Pipeline was used: Tokenizer → StopWordsRemover → HashingTF (10,000 features) → IDF (minDocFreq=2). The model was fit on a combined corpus (resumes + jobs) so IDF weights reflect overall term rarity across both domains.

Cosine similarity was implemented as a Spark UDF. A cross-join between the selected resume categories and all job postings produced 604,891 scored resume–job pairs. The top matches per resume were then ranked with a window function:

```
from pyspark.sql import Window
import pyspark.sql.functions as F

window = Window.partitionBy("resume_id").orderBy(F.desc("cosine_score"))
top_matches = (
    pairs
    .withColumn("rank", F.row_number().over(window))
    .filter(F.col("rank") <= 5)
)
```

Spark Structured Streaming (simulated intake)

To simulate "new resumes arriving," ten processed resumes were written as individual JSON files to a watched folder. Spark's readStream monitored that directory and ran foreachBatch every 10 seconds. Each micro-batch was vectorized using the same TF-IDF model, scored against the static job vectors, and the top 3 matches per resume were written to MongoDB (stream_results collection).

Results and Analysis

Match score overview

Across all scored pairs, cosine similarity ranged from 0.034 to 0.518, with a mean of 0.120. The scores are "low" in absolute terms, but that is expected here because most resumes in the dataset are from non-technical categories (e.g., healthcare, teaching, sales) while the job postings are all AI/data roles. The vocabulary overlap is genuinely limited.

ATS rejection simulation (thresholding)

To make the scores easier to interpret, I applied a simple threshold and treated anything below the cutoff as an automatic rejection. The pass/reject rates below show how sensitive the outcome is to the chosen threshold

Score threshold	Pass rate	Rejection rate
0.10	~72%	~28%
0.15	~38%	~62%
0.20	~16%	~84%
0.25	~5%	~95%

At a 0.20 cutoff, roughly 84% of resumes would be rejected before a human sees them. In this dataset, the main driver is vocabulary alignment: many resumes simply do not share enough job-description terminology to score well.

Keyword gap and length effects

A quick keyword-gap check compared common terms in job descriptions vs. resumes. The most underrepresented job terms in resumes included:

- machine learning
- python
- algorithms
- model
- neural
- pipeline

Resume length had near-zero relationship with score. Adding more text does not help by itself; the scoring responds to term overlap and weighting, not raw word count.

Category performance

Engineering and IT resumes scored highest on average, while categories like Healthcare, Chef, and Agriculture scored lowest. That aligns with intuition: the job dataset is entirely AI/data roles, so non-technical categories share very little vocabulary. If a candidate is applying outside their original sector, they often need to rewrite their resume using the target domain's language—not just list more experience.

Scalability Benchmarking

Two timing baselines were collected across 1×, 5×, and 10× dataset sizes: (A) the full Spark pipeline, and (B) a single-machine scikit-learn baseline for reference.

Scale	Resumes	Jobs	Load (s)	TF-IDF (s)	Match (s)	Engine
1×	2,483	735	3.53	14.95	80.16	Spark
5×	12,415	3,675	17.74	75.69	640.56	Spark
10×	24,830	7,350	37.47	150.46	2,267.25	Spark
1×	2,484	735	0.28	3.25	0.02	Python
5×	12,420	3,675	1.03	12.25	0.10	Python
10×	24,840	7,350	1.40	23.88	0.22	Python

Spark TF-IDF timings scale close to linearly (about 5× more time at 5× data, and about 10× at 10×), which is what you expect from the distributed implementation. The Python baseline looks faster here because the dataset is still small enough to fit comfortably in memory, and NumPy's optimized matrix operations avoid Spark's JVM and scheduling overhead. On a true multi-node cluster (not a single Colab runtime), Spark's match step would benefit much more from parallelism.

Pain Points and Caveats

Java compatibility

One early setup issue was Java versioning. The machine initially had Java 25 installed, but Spark 4.0 expects Java 8, 11, or 17. With Java 25, the Spark session would hang without a helpful error. Installing Java 11 (Eclipse Adoptium) and setting JAVA_HOME fixed it.

Colab session limits

Colab's free tier can disconnect and clear state if the browser goes idle. Because the cross-join matching step runs for several minutes, a disconnect meant rerunning expensive cells. Checkpointing to MongoDB and Parquet after each major stage made re-runs manageable.

MongoDB URI escaping

MongoDB Atlas authentication can break if the password contains special characters (e.g., @ or #) because they have meaning in a URI. URL-encoding the password (urllib.parse.quote_plus) prevented parsing errors. Colab's changing IPs also required permissive network access settings.

Dataset limitation: single job family

All 735 job postings are in the same job_family ("data scientist") because they were scraped using a single search term. That makes category-to-category matching less meaningful and pulls down the overall mean score for non-technical resume categories.

Cross-join benchmarking

Running the full cross-join inside the scaling loop was not practical (it timed out after ~50+ minutes). Benchmarking was adjusted so TF-IDF was measured on the full scaled corpora, while the match step used a smaller sample for timing. The full 604,891-pair run was still executed and validated in the main pipeline.

Requirements Checklist

The table below maps the project components to the course requirements.

Requirement	Status	Evidence
Apache Spark- mandatory	✓ Done	Cells 28–42, Spark 4.0.2
Second big data tool (MongoDB)	✓ Done	Cells 22–26, Atlas M0
Spark Structured Streaming	✓ Done	Cells 64–73
Real-world datasets	✓ Done	2,484 resumes + 735 jobs (Kaggle)
Data cleaning and preprocessing	✓ Done	Cells 17–19, 30–31
Aggregations / window functions	✓ Done	Token stats, category ranking, window rank
At least one analytical task	✓ Done	5 analyses: rejection sim, keyword gap, length vs score, category rank, report card
Results stored in files/databases	✓ Done	6 MongoDB collections + 3 Parquet files
Visualizations	✓ Done	8 matplotlib charts saved to outputs/charts/
Scalability benchmarking	✓ Done	1×/5×/10× PySpark + Python baseline
Source code submitted	✓ Done	ATS_Automation_Project.ipynb

Conclusion

This project implemented a working ATS-style matching pipeline using Apache Spark, MongoDB Atlas, and Spark Structured Streaming. Across 604,891 resume–job pairs, the system produced similarity scores, stored structured outputs for querying, and supported a simple streaming simulation for "new resume" intake.

On the engineering side, the work reinforced a few practical lessons: Spark's overhead is real in single-machine environments, checkpointing saves your sanity in Colab, and clean schemas make MongoDB storage straightforward. On the analysis side, the threshold simulation makes the main point: small changes in a cutoff can move a large fraction of applicants from "seen" to "rejected."

The biggest limitation is the job-posting dataset: it covers only one job family (data scientist), which skews results for non-technical resume categories. A next step would be to expand job postings across multiple sectors and repeat the same matching + threshold analysis to see how outcomes change.

Bottom line: in this simplified setup, ATS filtering behaves like a vocabulary-alignment problem more than a "how much experience did you write down" problem. If the words in the resume do not line up with the words in the job description, the math will usually treat it as a weak match.

References

Apache Software Foundation. (2024). Apache Spark Unified Analytics Engine for Big Data. https://spark.apache.org

Apache Software Foundation. (2024). Spark MLlib: Machine Learning Library. https://spark.apache.org/mllib

Kaggle. (2023). Resume Dataset (snehaanbhawal). https://www.kaggle.com/datasets/snehaanbhawal/resume-dataset

Kaggle. (2024). AI and Data Jobs Dataset 2025 (princekhunt19). https://www.kaggle.com/datasets/princekhunt19/700-jobs-data-of-ai-and-data-fields-2025

MongoDB. (2024). MongoDB Atlas Documentation. https://www.mongodb.com/docs/atlas